Ramageddon

https://github.com/Oimeow/174AFinalProject
Damian Toma (dtoma@ucla.edu | 606964154 | Oimeow)
Jay Thapar (jayt02@ucla.edu | 806679434 | Jay-Thpr)
Kevin Wong (rectangles@ucla.edu | 806222091 | kwong05)

Theme/story

The year is 2028. Elon Musk is the world's first quadrillionaire and Sam Altman has stolen all of the RAM. We must fulfill interplanetary tasks to make ends meet and build our new gaming PC.

Players will visit planets by ship (restricted by fuel) and complete tasks by foot

- Slingshot between planets

Players will have a home base where they can start tasks. They will complete missions on different planets to collect RAM components, which they bring back to assemble their PC. Each trip requires planning around limited fuel and using gravitational slingshots.

Class topics covered

Transformation Matrices: Translate, rotate, and scale: planets, spaceships, planets, and terrain

Camera Transformation: Transition between third-person spaceship exploration, first-person inner-planet camera, third-person slingshot view

Lighting and Shading: Simulate illuminations from the star, planetary day/night cycles, material lighting

Geometry and Meshes: Generate terrain meshes and planetary surfaces with different levels of detail

Animation: Animate planetary orbits, spaceship movement, and objects in the environment

Interactivity elements

Mouse controls for launching spaceship + **mouse pointer** for aiming and looking
WASD for movement on planets, **space** to jump
E to interact
(maybe) Click to drop/throw explosive*

Advanced features

Physics-Based Gravity Simulation

- Simulate gravitational forces between the spaceship and planets using Newton's law of gravity. Numerically integrate the spaceship's velocity and position over time, allowing players to slingshot around planets using gravitational assists.

Collision Detection (Guide users to planets, Determine first- or third-person, Handle walking on surfaces and obstacles while in first-person)

- Approximate planets as spheres and the spaceship/player using simple bounding volumes. Detect when the spaceship intersects planetary surfaces to trigger landing transitions, and prevent players from walking through terrain.

Procedural Generation of Worlds

- Generate planetary terrain using heightmaps or noise functions applied to a mesh, with varying terrain parameters for different planets. Use lower-resolution meshes when planets are viewed from space.

Trajectory Prediction

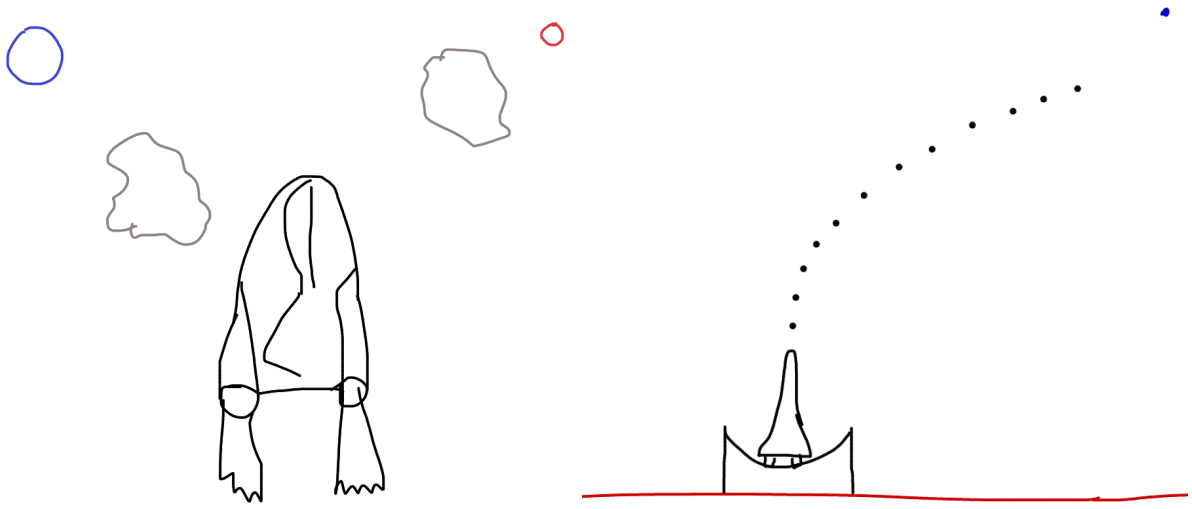
- Simulate a short sequence of future spaceship positions under gravity and render a predicted trajectory to help players aim their launches.

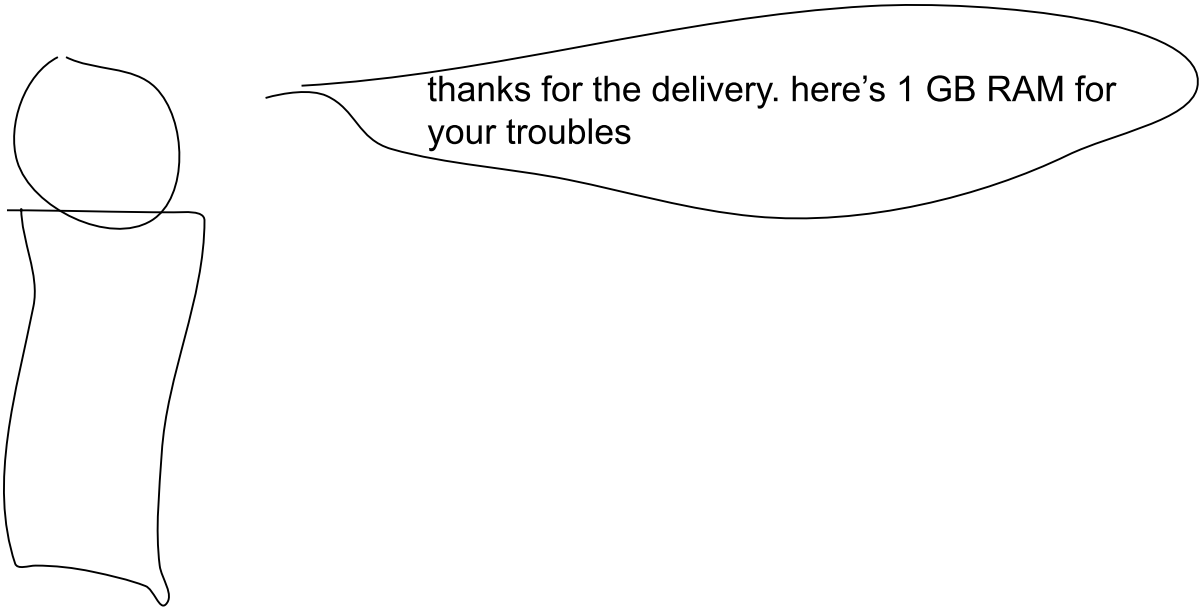
Artificial Intelligence companions*

Explosive Forces*

*may be implemented later if time is available

Prototype visual:





(Press E to interact)